



СБОРНИК ЗАДАНИЙ И РЕШЕНИЙ

Python — Часть 4: Специализация

Regex • Скрапинг • Flask • Pandas • Matplotlib • Telegram • Tkinter

☆ Лёгкие

★★ Средние

★★★ Сложные

⚠ Важно: Некоторые задания требуют установки библиотек: `pip install flask pandas matplotlib beautifulsoup4 requests aiogram`. Задания по Tkinter и Flask запускать локально.

Тема 23: Регулярные выражения (re)

Регулярные выражения — мощный инструмент поиска и замены по шаблону. Один из главных навыков разработчика.

Задание 23.1: Валидатор данных

☆☆☆

Напиши три функции-валидатора:

- `is_valid_email(s)` — проверяет email
- `is_valid_phone(s)` — российский номер +7 (XXX) XXX-XX-XX
- `is_valid_date(s)` — дата в формате ДД.ММ.ГГГГ

Проверь каждую на 3 корректных и 3 некорректных примерах.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import re</code>	Модуль регулярных выражений
<code>def is_valid_email(s):</code>	
<code> pattern = r'^[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}\$'</code>	<code>r</code> — raw-строка: \ не экранируется
<code> return bool(re.match(pattern, s))</code>	<code>re.match</code> проверяет с начала строки
	<code>^</code> — начало, <code>\$</code> — конец строки
<code>def is_valid_phone(s):</code>	
<code> pattern = r'^\+7 \(\d{3}\) \d{3}-\d{2}-\d{2}\$'</code>	<code>\d{3}</code> — ровно 3 цифры
<code> return bool(re.match(pattern, s))</code>	<code>\(\) </code> — буквальныe скобки
<code>def is_valid_date(s):</code>	
<code> pattern = r'^\d{2}\.\d{2}\.\d{4}\$'</code>	<code>.</code> — буквальная точка (не любой символ)
<code> return bool(re.match(pattern, s))</code>	
<code># Тесты email</code>	
<code>emails = ['user@mail.ru',</code>	
<code>'test+1@sub.domain.com', 'bad@',</code>	
<code> 'no-at-sign', '@nodomain.com',</code>	
<code>'user@.ru']</code>	
<code>for e in emails:</code>	
<code> print(f"{'OK' if is_valid_email(e) else 'FAIL':4} {e}")</code>	<code>f</code> -строка с условием прямо внутри <code>{}</code>
<code># Тесты телефонов</code>	
<code>phones = ['+7 (495) 123-45-67', '+7 (800)</code>	
<code>555-35-35',</code>	
<code> '89161234567', '7 (495) 123-45-</code>	
<code>67', '+8 (495) 123-45-67']</code>	
<code>for ph in phones:</code>	
<code> print(f"{'OK' if is_valid_phone(ph)</code>	
<code>else 'FAIL':4} {ph}")</code>	

► **Результат:**

```
OK    user@mail.ru
OK    test+1@sub.domain.com
FAIL  bad@
FAIL  no-at-sign
FAIL  @nodomain.com
FAIL  user@.ru
OK    +7 (495) 123-45-67
OK    +7 (800) 555-35-35
FAIL  89161234567
FAIL  7 (495) 123-45-67
```

⊖ **Частая ошибка:**

✗ `pattern = '.*@.*\..*'` # слишком широко

✓ `pattern = r'^[\w.+-]+@[ \w-]+\.[a-zA-Z]{2,}$'`

Шаблон `.*@.*\..*` пропускает почти всё, включая некорректные адреса. Регулярки нужно делать достаточно строгими — иначе польза от валидации минимальна.

Задание 23.2: Парсер логов

★★☆

Дан текст лога сервера. Напиши функцию `parse_log(text)`, которая:

- находит все строки формата: [УРОВЕНЬ] ДД.ММ.ГГГГ ЧЧ:ММ:СС сообщение
- возвращает список словарей `{level, date, time, message}`
- функцию `filter_by_level(logs, level)` — фильтр по уровню

Тестовый лог предоставлен в решении.

<code>import re</code>	
<code>LOG_TEXT = '''</code>	Тестовый лог — тройные кавычки
<code>[INFO] 01.03.2025 10:00:01 Сервер запущен</code>	
<code>[ERROR] 01.03.2025 10:05:22 Нет подключения к БД</code>	
<code>[WARN] 01.03.2025 10:05:23 Повторная попытка...</code>	
<code>[INFO] 01.03.2025 10:05:30 Подключение восстановлено</code>	
<code>[ERROR] 01.03.2025 11:30:00 Ошибка авторизации</code>	
<code>'''</code>	
<code>def parse_log(text):</code>	
<code> pattern =</code> <code>r'\[(\w+)\]\s+(\d{2}\.\d{2}\.\d{4})\s+(\d{2}:\d{2}:\d{2})\s+(.+)'</code>	4 группы захвата в скобках
<code> results = []</code>	
<code> for match in re.finditer(pattern, text):</code>	finditer — итератор по всем совпадениям
<code> results.append({</code>	
<code> 'level': match.group(1),</code>	group(1) — первая группа захвата
<code> 'date': match.group(2),</code>	group(2) — вторая группа
<code> 'time': match.group(3),</code>	
<code> 'message': match.group(4).strip()</code>	group(4) — текст сообщения
<code> })</code>	
<code> return results</code>	
<code>def filter_by_level(logs, level):</code>	
<code> return [l for l in logs if l['level'] == level]</code>	List comprehension с условием
<code>logs = parse_log(LOG_TEXT)</code>	Парсим лог
<code>print(f'Всего записей: {len(logs)}')</code>	
<code>errors = filter_by_level(logs, 'ERROR')</code>	Фильтруем ошибки
<code>print(f'Ошибок: {len(errors)}')</code>	
<code>for e in errors:</code>	
<code> print(f" {e['date']} {e['time']} — {e['message']}")</code>	

► Результат:

Всего записей: 5

Ошибок: 2

01.03.2025 10:05:22 — Нет подключения к БД

01.03.2025 11:30:00 — Ошибка авторизации

Задание 23.3: Обработчик текста

★★★

Напиши функцию `clean_text(text)`, которая:

1. Убирает HTML-теги (`<b>`, `</p>`, и т.д.)
 2. Заменяет множественные пробелы/переносы на один пробел
 3. Маскирует номера карт (16 цифр → `**** * X`)
 4. Выделяет все URL в список
- Верни (очищенный_текст, список_url).

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

import re	
def clean_text(text):	
# 1. Убираем HTML-теги	
text = re.sub(r'<[^>]+>', '', text)	<i><[^>]+> — теги: < затем не->, затем ></i>
# 2. Нормализуем пробелы	
text = re.sub(r'\s+', ' ', text).strip()	<i>\s+ — один или более пробельных символов</i>
# 3. Маскируем карты	
def mask_card(m):	<i>Функция замены для re.sub</i>
digits = re.sub(r'\D', '', m.group())	<i>Оставляем только цифры</i>
return f'**** *{digits[-4:]}'	<i>Показываем только последние 4</i>
text = re.sub(r'\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b',	<i>16 цифр с возможными пробелами/дефисами</i>
mask_card, text)	<i>mask_card — callback-функция</i>
# 4. Извлекаем URL	
urls = re.findall(r'https?://[^\s<>"]+', text)	<i>https?:// — http или https, затем до пробела</i>
text = re.sub(r'https?://[^\s<>"]+', '[ссылка]', text)	<i>Заменяем URL на плейсхолдер</i>
return text.strip(), urls	<i>Возвращаем пару (текст, ссылки)</i>
sample = ''' Новость	
<p>Оплата картой 4111 1111 1111 1234	
прошла.</p>	
Подробнее: https://news.example.com/article	
Ещё: https://python.org'''	
clean, urls = clean_text(sample)	
print('Текст:', clean)	
print('URLs:', urls)	

► Результат:

Текст: Новость Оплата картой **** * 1234 прошла. Подробнее: [ссылка]
 Ещё: [ссылка]
 URLs: ['https://news.example.com/article', 'https://python.org']

□ Тест: Тема 23

? Вопрос: Чем `re.match()` отличается от `re.search()`?

А) Ничем

Б) `match` проверяет только с начала строки ← *правильный ответ*

В) `search` быстрее

Г) `match` возвращает список

? Вопрос: Что означает `r"` перед строкой шаблона?

А) Рекурсивный шаблон

Б) Raw-строка: обратные слешы не экранируются ← *правильный ответ*

В) Регистронезависимый поиск

Г) Режим `replace`

? Вопрос: Как найти ВСЕ совпадения в строке?

А) `re.match()`

Б) `re.search()`

В) `re.findall()` ← *правильный ответ*

Г) `re.compile()`

Тема 24: Веб-скрапинг (BeautifulSoup)

Веб-скрапинг — автоматический сбор данных с сайтов. Используй только на сайтах, которые это разрешают.

Задание 24.1: Парсинг учебного сайта

☆☆☆

Напиши скрапер для `http://quotes.toscrape.com` (сайт создан специально для практики). Извлеки с первой страницы:

- текст каждой цитаты
- имя автора
- теги

Выведи первые 3 цитаты. Сохрани все в список словарей.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import requests</code>	<i>pip install requests beautifulsoup4</i>
<code>from bs4 import BeautifulSoup</code>	
<code>URL = 'http://quotes.toscrape.com'</code>	<i>Учебный сайт для практики скрапинга</i>
<code>def get_quotes(url):</code>	
<code> headers = {'User-Agent': 'Mozilla/5.0'}</code>	<i>User-Agent — представляемся браузером</i>
<code> response = requests.get(url,</code> <code>headers=headers, timeout=10)</code>	<i>timeout=10 — ждём не дольше 10 сек</i>
<code> response.raise_for_status()</code>	<i>Бросает исключение если код != 200</i>
<code> soup = BeautifulSoup(response.text,</code> <code>'html.parser')</code>	<i>html.parser — встроенный парсер Python</i>
<code> quotes = []</code>	
<code> for block in soup.find_all('div',</code> <code>class_='quote'):</code>	<i>find_all — ищет все теги с классом quote</i>
<code> text = block.find('span',</code> <code>class_='text').get_text()</code>	<i>get_text() — текст без HTML-тегов</i>
<code> author = block.find('small',</code> <code>class_='author').get_text()</code>	
<code> tags = [t.get_text() for t in</code> <code>block.find_all('a', class_='tag')]</code>	<i>Список тегов через list comprehension</i>
<code> quotes.append({'text': text,</code> <code>'author': author, 'tags': tags})</code>	<i>Сохраняем как словарь</i>
<code> return quotes</code>	
<code>quotes = get_quotes(URL)</code>	
<code>print(f'Найдено цитат: {len(quotes)}')</code>	
<code>print()</code>	
<code>for q in quotes[:3]:</code>	<i>Первые 3</i>
<code> print(f'Автор: {q["author"]}') print(f'Теги: {' , '.join(q["tags"])}')</code>	
<code> print(f'Текст: {q["text"][:80]}...')</code>	<i>Первые 80 символов</i>
<code> print('-' * 50)</code>	

► **Результат:**

Найдено цитат: 10

Автор: Albert Einstein

Теги: change, deep-thoughts, thinking

Текст: "The world as we have created it is a proce...

...

⊖ **Частая ошибка:**

✗ `soup.find('div.quote')`

✓ `soup.find('div', class_='quote')`

CSS-селектор 'div.quote' не работает в `find()`. Нужно передавать тег и класс отдельными аргументами. Для CSS-селекторов используй `soup.select('div.quote')`.

Задание 24.2: Многостраничный скрапер

★★★

Расширь скрапер: обходи все страницы сайта `quotes.toscrape.com` (их 10).

Добавь паузу 0.5 сек между запросами.

Сохрани все цитаты в CSV-файл с полями: author, text, tags.

Подсчитай топ-5 авторов по количеству цитат.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

<code>import requests, csv, time</code>	
<code>from bs4 import BeautifulSoup</code>	
<code>from collections import Counter</code>	Counter — удобный счётчик
<code>BASE = 'http://quotes.toscrape.com'</code>	Базовый URL
<code>def scrape_all_pages():</code>	
<code>all_quotes = []</code>	Список для всех цитат
<code>page = 1</code>	Начинаем с первой страницы
<code>while True:</code>	Продолжаем пока есть страницы
<code>url = f'{BASE}/page/{page}/'</code>	Формируем URL страницы
<code>print(f'Страница {page}...')</code>	
<code>r = requests.get(url,</code> <code>headers={'User-Agent': 'Mozilla/5.0'},</code> <code>timeout=10)</code>	
<code>soup = BeautifulSoup(r.text,</code> <code>'html.parser')</code>	
<code>blocks = soup.find_all('div',</code> <code>class_='quote')</code>	Блоки цитат на текущей странице
<code>if not blocks:</code>	Нет блоков — страниц больше нет
<code>break</code>	Выходим из цикла
<code>for b in blocks:</code>	

<code>all_quotes.append({</code>	
<code> 'author': b.find('small',</code> <code>class_='author').get_text(),</code>	
<code> 'text': b.find('span',</code> <code>class_='text').get_text(),</code>	
<code> 'tags':</code> <code>' '.join(t.get_text() for t in</code> <code>b.find_all('a', class_='tag'))</code>	Теги через — удобно для CSV
<code> })</code>	
<code>page += 1</code>	Следующая страница
<code>time.sleep(0.5)</code>	Пауза — не перегружаем сервер!
<code>return all_quotes</code>	
<code>quotes = scrape_all_pages()</code>	
<code>print(f'Всего цитат: {len(quotes)}')</code>	
<code># Сохраняем в CSV</code>	
<code>with open('quotes.csv', 'w', newline='',</code> <code>encoding='utf-8') as f:</code>	
<code> writer = csv.DictWriter(f,</code> <code>fieldnames=['author', 'text', 'tags'])</code>	
<code> writer.writeheader()</code>	
<code> writer.writerows(quotes)</code>	
<code>print('Сохранено в quotes.csv')</code>	
<code># Топ-5 авторов</code>	
<code>authors = Counter(q['author'] for q in</code> <code>quotes)</code>	Counter подсчитывает вхождения
<code>print('Топ-5 авторов:')</code>	
<code>for author, count in</code> <code>authors.most_common(5):</code>	.most_common(N) — топ N по частоте
<code> print(f' {author}: {count} цитат')</code>	

► Результат:

```

Страница 1...
Страница 2...
...
Всего цитат: 100
Сохранено в quotes.csv
Топ-5 авторов:
  Albert Einstein: 10 цитат
  J.K. Rowling: 7 цитат
  ...

```

□ Тест: Тема 24

? Вопрос: Что делает `response.raise_for_status()`?

А) Выводит статус-код

Б) Бросает исключение если статус не 2xx ← *правильный ответ*

В) Повторяет запрос

Г) Возвращает текст ответа

? Вопрос: Зачем нужна пауза `time.sleep()` между запросами?

А) Для красоты кода

Б) Чтобы не перегружать сервер и не быть заблокированным ← *правильный ответ*

В) Требование BeautifulSoup

Г) Для сохранения в CSV

Тема 25: Веб-разработка (Flask)

Flask — минималистичный веб-фреймворк. Позволяет создать рабочий сайт за несколько строк кода.

Задание 25.1: Персональная страница

☆☆☆

Создай Flask-приложение с двумя маршрутами:
— / — главная страница с твоим именем и фото (можно placeholder)
— /about — страница «О себе» с кратким описанием
Добавь навигацию между страницами.
Применни базовую CSS-стилизацию прямо в HTML.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>from flask import Flask, render_template_string</code>	<i>render_template_string — HTML прямо в коде</i>
<code>app = Flask(__name__)</code>	<i>__name__ — имя текущего модуля</i>
<code>BASE_HTML = ''' <!DOCTYPE html> <html lang='ru'> <head> <meta charset='UTF-8'> <title>{{ title }}</title> <style> body { font-family: Arial; max-width: 800px; margin: 0 auto; padding: 20px; } nav a { margin-right: 15px; color: steelblue; text-decoration: none; } nav a:hover { text-decoration: underline; } </style> </head> <body> <nav>ГлавнаяО себе</nav> <hr> {% block content %}{% endblock %} </body></html>'''</code>	<i>Базовый шаблон с навигацией</i>
<code>INDEX_HTML = BASE_HTML.replace('{% block content %}{% endblock %}', '''<h1>Привет! Я {{ name }}</h1> </code>	<i>{{ }} — переменные Jinja2</i>
	<i>block — место для контента страницы</i>
	<i>Используем replace вместо наследования</i>
	<i>placeholder-изображение</i>

<code><p>Изучаю Python и веб-разработку!</p>'''</code>	
<code>)</code>	
<code>@app.route('/')</code>	<i>Декоратор регистрирует маршрут</i>
<code>def index():</code>	
<code> return render_template_string(INDEX_HTML, title='Главная', name='Алёша')</code>	<i>Передаём переменные в шаблон</i>
<code>@app.route('/about')</code>	
<code>def about():</code>	
<code> return render_template_string(BASE_HTML.replace('{% block content %}{% endblock %}', '<h1>О себе</h1><p>Мне 16 лет, учусь в 9 классе.</p>), title='О себе')</code>	
<code>if __name__ == '__main__':</code>	
<code> app.run(debug=True)</code>	<i>debug=True — автоперезапуск при изменениях</i>

❑ **Как запустить:** Сохрани как app.py, выполни `python app.py`. Открой браузер: `http://127.0.0.1:5000`. Видишь свою страницу!

Задание 25.2: REST API для списка задач

★★★

Создай Flask REST API для управления задачами (To-Do).

Маршруты:

- GET /api/tasks — все задачи
- POST /api/tasks — добавить задачу
- PUT /api/tasks/<id> — обновить статус
- DELETE /api/tasks/<id> — удалить

Данные храни в памяти (список). Возвращай JSON.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

```
from flask import Flask, jsonify, request, abort
```

jsonify — словарь → JSON-ответ

<code>app = Flask(__name__)</code>	
<code>tasks = []</code>	Хранилище задач в памяти
<code>next_id = 1</code>	Счётчик ID — глобальная переменная
<code>@app.route('/api/tasks', methods=['GET'])</code>	<code>methods=['GET']</code> — только GET-запросы
<code>def get_tasks():</code>	
<code> return jsonify(tasks)</code>	<code>jsonify</code> превращает список в JSON-ответ
<code>@app.route('/api/tasks', methods=['POST'])</code>	POST — создание новой задачи
<code>def create_task():</code>	
<code> global next_id</code>	<code>global</code> нужен чтобы изменить глобальную
<code> data = request.get_json()</code>	<code>get_json()</code> парсит тело запроса
<code> if not data or 'title' not in data:</code>	Проверяем наличие обязательного поля
<code> abort(400)</code>	400 Bad Request — неверный запрос
<code> task = {'id': next_id, 'title': data['title'], 'done': False}</code>	Создаём задачу
<code> tasks.append(task)</code>	
<code> next_id += 1</code>	
<code> return jsonify(task), 201</code>	201 Created — успешное создание
<code>@app.route('/api/tasks/<int:task_id>', methods=['PUT'])</code>	<code><int:task_id></code> — параметр URL как int
<code>def update_task(task_id):</code>	<code>task_id</code> передаётся автоматически
<code> task = next((t for t in tasks if t['id'] == task_id), None)</code>	<code>next()</code> с generator expr — поиск по id
<code> if task is None:</code>	
<code> abort(404)</code>	404 Not Found
<code> task['done'] = not task['done']</code>	Переключаем статус
<code> return jsonify(task)</code>	
<code>@app.route('/api/tasks/<int:task_id>', methods=['DELETE'])</code>	
<code>def delete_task(task_id):</code>	
<code> global tasks</code>	
<code> original_len = len(tasks)</code>	Запоминаем длину до удаления
<code> tasks = [t for t in tasks if t['id'] != task_id]</code>	Фильтруем — убираем нужный id
<code> if len(tasks) == original_len:</code>	Если длина не изменилась — не нашли
<code> abort(404)</code>	
<code> return jsonify({'status': 'deleted'})</code>	
<code>if __name__ == '__main__':</code>	
<code> app.run(debug=True)</code>	

► Результат:

```
# GET http://localhost:5000/api/tasks
[]
```

```
# POST с телом {"title": "Выучить Flask"}
{"done": false, "id": 1, "title": "Выучить Flask"}

# GET снова
[{"done": false, "id": 1, "title": "Выучить Flask"}]
```

□ Тест: Тема 25

? Вопрос: Какой HTTP-статус означает 'ресурс успешно создан'?

А) 200 OK

Б) 201 Created ← *правильный ответ*

В) 204 No Content

Г) 202 Accepted

? Вопрос: Как Flask получает JSON из тела POST-запроса?

А) request.json()

Б) request.body

В) request.get_json() ← *правильный ответ*

Г) request.data.json()

Тема 26: Анализ данных (Pandas)

Pandas — главная библиотека для анализа данных. DataFrame — таблица на стероидах

Задание 26.1: Первый DataFrame

☆☆☆

Создай DataFrame из словаря с данными о 5 студентах:
колонки: имя, класс, математика, физика, история.
Выведи:
— первые 3 строки (`.head(3)`)
— базовую статистику (`.describe()`)
— средний балл по каждому предмету
— добавь колонку 'средний' и отсортируй по ней.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import pandas as pd</code>	Стандартный псевдоним <code>pd</code>
<code>data = {</code>	Словарь — основа <code>DataFrame</code>
<code> 'имя':</code>	
<code> ['Алёша', 'Маша', 'Дима', 'Катя', 'Вова'],</code>	
<code> 'класс':</code>	
<code> ['9А', '9В', '9А', '9В', '9А'],</code>	
<code> 'математика': [95, 78, 65, 91, 55],</code>	
<code> 'физика':</code>	
<code> [88, 92, 70, 87, 60],</code>	
<code> 'история':</code>	
<code> [72, 85, 88, 94, 72],</code>	
<code>}</code>	
<code>df = pd.DataFrame(data)</code>	Создаём <code>DataFrame</code> из словаря
<code>print('--- Первые 3 строки ---')</code>	
<code>print(df.head(3))</code>	<code>.head(n)</code> — первые <code>n</code> строк
<code>print('--- Статистика ---')</code>	
<code>print(df.describe())</code>	<code>.describe()</code> — count, mean, std, min, max...
<code>print('--- Средний балл по предметам ---')</code>	
<code>subjects = ['математика', 'физика',</code>	
<code> 'история']</code>	
<code>print(df[subjects].mean())</code>	<code>df[список]</code> выбирает несколько колонок
<code># Добавляем колонку среднего</code>	
<code>df['средний'] = df[subjects].mean(axis=1)</code>	<code>axis=1</code> — среднее по строке (а не столбцу)
<code>df['средний'] = df['средний'].round(1)</code>	Округляем до 1 знака
<code># Сортируем по среднему (убывание)</code>	
<code>df_sorted = df.sort_values('средний',</code>	<code>ascending=False</code> — от большего к меньшему
<code> ascending=False)</code>	
<code>print(df_sorted[['имя', 'средний']])</code>	Выводим только нужные колонки

► **Результат:**

```
--- Первые 3 строки ---
   имя класс  математика  физика  история
0  Алёша    9А           95      88      72
...
--- Рейтинг ---
   имя  средний
3  Катя    90.7
0  Алёша   85.0
1  Маша    85.0
```

Задание 26.2: Анализ продаж

★★★

Создай DataFrame с данными о продажах за 12 месяцев:

колонки: месяц, продукт_А, продукт_Б, расходы.

Выполни анализ:

- добавь колонку 'прибыль' = (А + Б) - расходы
- найди самый прибыльный и убыточный месяц
- сгруппируй по кварталам и посчитай суммарную прибыль
- найди месяцы, где прибыль выше средней

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

```
import pandas as pd
```

```
import numpy as np
```

numpy для генерации данных

```
months =
['Янв', 'Фев', 'Мар', 'Апр', 'Май', 'Июн',
```

```
'Июл', 'Авг', 'Сен', 'Окт', 'Ноя', 'Дек']
```

```
np.random.seed(42)
```

seed — фиксируем случайность для повторяемости

```
df = pd.DataFrame({
```

```
    'месяц':      months,
```

```
    'продукт_А': np.random.randint(100,
300, 12),
```

randint(min, max, кол-во) — случайные int

```
    'продукт_Б': np.random.randint(80,
250, 12),
```

```
    'расходы':   np.random.randint(150,
400, 12),
```

```
})
```

```
# Прибыль
```

```
df['выручка'] = df['продукт_А'] +
df['продукт_Б']
```

Новая колонка = арифметика других колонок

```
df['прибыль'] = df['выручка'] -
df['расходы']
```

# Лучший и худший месяц	
best = df.loc[df['прибыль'].idxmax(), 'месяц']	<i>.idxmax() — индекс максимального значения</i>
worst = df.loc[df['прибыль'].idxmin(), 'месяц']	<i>.idxmin() — индекс минимального значения</i>
print(f'Лучший: {best}, худший: {worst}')	
# Кварталы	
df['квартал'] = pd.cut(df.index,	<i>pd.cut — делит на интервалы</i>
bins=[0,3,6,9,12],	<i>Границы кварталов (0-3, 3-6, 6-9, 9-12)</i>
labels=['Q1','Q2','Q3','Q4'])	<i>Метки кварталов</i>
quarterly = df.groupby('квартал',	<i>.groupby().sum() — суммируем по группам</i>
observed=True) ['прибыль'].sum()	
print('Прибыль по кварталам:')	
print(quarterly)	
# Месяцы выше среднего	
avg = df['прибыль'].mean()	<i>.mean() — среднее значение</i>
above_avg = df[df['прибыль'] >	<i>Булева маска — фильтрация строк</i>
avg][['месяц', 'прибыль']]	
print(f'Выше среднего ({avg:.0f}):')	
print(above_avg.to_string(index=False))	<i>to_string(index=False) — без индексов</i>

► Результат:

```

Лучший: Мар, худший: Фев
Прибыль по кварталам:
квартал
Q1    -23
Q2   187
Q3   234
Q4   156
Выше среднего (138):
  месяц  прибыль
  Мар      245
  ...

```

□ Тест: Тема 26

? Вопрос: Что делает `df[df['score'] > 80]`?

А) Удаляет строки со `score <= 80`

Б) Возвращает DataFrame только со строками где `score > 80` ← *правильный ответ*

В) Сортирует по `score`

Г) Подсчитывает строки

? Вопрос: Чем отличается `axis=0` от `axis=1` в `.mean()`?

А) Ничем

Б) `axis=0` — среднее по столбцам, `axis=1` — по строкам ← *правильный ответ*

В) `axis=1` — по столбцам, `axis=0` — по строкам

Г) `axis` задаёт точность

Тема 27: Визуализация данных (Matplotlib)

Хороший график говорит больше тысячи чисел. Matplotlib — стандарт визуализации в Python.

Задание 27.1: Четыре типа графиков

☆☆☆

Создай фигуру с 4 подграфиками (2x2):
— линейный: динамика температуры за неделю
— столбчатый: продажи по дням
— круговой: доли рынка компаний
— точечный: рост vs вес 20 человек (случайные данные)
Добавь заголовок, подписи осей, легенду где нужно.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

```
import matplotlib.pyplot as plt
```

pip install matplotlib

```
import numpy as np
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
```

2x2 подграфика, размер 12x8 дюймов

```
fig.suptitle('Сборник графиков', fontsize=16, fontweight='bold')
```

Общий заголовок для всей фигуры

```
# 1. Линейный — температура
```

```
days = ['Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб', 'Вс']
```

```
temps = [12, 15, 11, 18, 22, 25, 20]
```

```
axes[0,0].plot(days, temps, 'o-', color='tomato', linewidth=2)
```

axes[строка, столбец] — выбор подграфика

```
axes[0,0].set_title('Температура (°C)')
```

```
axes[0,0].set_ylabel('Температура')
```

```
axes[0,0].grid(alpha=0.3)
```

Сетка с прозрачностью 30%

```
axes[0,0].fill_between(days, temps, alpha=0.1, color='tomato')
```

Заливка под графиком

```
# 2. Столбчатый — продажи
```

```
sales = [45, 60, 35, 70, 85, 90, 55]
```

```
bars = axes[0,1].bar(days, sales, color='steelblue', edgecolor='white')
```

bar() — столбчатый график

```
axes[0,1].set_title('Продажи по дням')
```

```
axes[0,1].bar_label(bars)
```

bar_label — подписи значений на столбцах

```
# 3. Круговой — доли
```

```
companies = ['Apple', 'Samsung', 'Xiaomi', 'Другие']
```

```
shares = [28, 22, 13, 37]
```

```
axes[1,0].pie(shares, labels=companies, autopct='%1.1f%%',
```

autopct — формат процентов

<code>colors=['#4ECDC4', '#45B7D1', '#96CEB4', '#FFEAA7']</code>	Цвета для каждого сектора
<code>axes[1,0].set_title('Доли рынка')</code>	
<code># 4. Точечный – рост/вес</code>	
<code>np.random.seed(0)</code>	Фиксируем случайность
<code>height = np.random.normal(170, 10, 20)</code>	normal(среднее, std, кол-во)
<code>weight = height * 0.45 + np.random.normal(0, 5, 20)</code>	Линейная зависимость + шум
<code>axes[1,1].scatter(height, weight, color='purple', alpha=0.7, s=80)</code>	s=80 — размер точек
<code>axes[1,1].set_title('Рост vs Вес')</code>	
<code>axes[1,1].set_xlabel('Рост (см)')</code>	
<code>axes[1,1].set_ylabel('Вес (кг)')</code>	
<code>plt.tight_layout()</code>	Автоматически подгоняет отступы
<code>plt.savefig('charts.png', dpi=150, bbox_inches='tight')</code>	dpi=150 — качество, bbox_inches — обрезка
<code>plt.show()</code>	Показываем окно с графиком

□ **Результат:** Файл charts.png с четырьмя графиками. При запуске также открывается интерактивное окно.

⊖ Частая ошибка:

✗ `plt.title('Заголовок')` # для subplots

✓ `axes[0,0].set_title('Заголовок')` # для каждого подграфика

При использовании `subplots()` нужно вызывать методы на объекте `axes`, а не на `plt` напрямую. `plt.title()` задаёт заголовок только для текущего (последнего) активного графика.

□ Тест: Тема 27

? **Вопрос:** Что делает `plt.tight_layout()`?

А) Сжимает график до минимального размера

Б) Автоматически подгоняет отступы между подграфиками ← *правильный ответ*

В) Сохраняет файл

Г) Показывает окно

? **Вопрос:** Какой тип графика лучше всего показывает доли целого?

А) Линейный

Б) Столбчатый

В) Круговой ← *правильный ответ*

Г) Точечный

Тема 28: Telegram-боты (aiogram)

Telegram-бот — программа, которая общается через мессенджер. Для работы нужен токен от @BotFather.

Задание 28.1: Эхо-бот с командами

☆☆☆

Создай Telegram-бота, который:

- на /start — приветствует по имени
- на /help — показывает список команд
- на /time — присылает текущее время
- на любое сообщение — отправляет его обратно (эхо)

Токен храни в переменной TOKEN.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

# pip install aiogram	Установка библиотеки
import asyncio	aiogram работает асинхронно
from datetime import datetime	
from aiogram import Bot, Dispatcher, types	Основные классы aiogram
from aiogram.filters import Command	Фильтр для команд /start, /help...
TOKEN = 'ВАШ_ТОКЕН_ОТ_BOTFATHER'	Замени на реальный токен!
bot = Bot(token=TOKEN)	Объект бота
dp = Dispatcher()	Диспетчер — маршрутизирует сообщения
@dp.message(Command('start'))	Декоратор: обрабатывать команду /start
async def start(message: types.Message):	async def — асинхронная функция
name = message.from_user.first_name	Имя пользователя из Telegram
await message.answer(	await — ждём ответа от Telegram API
f'Привет, {name}! Я готов к работе.\n'	
f'Введи /help чтобы узнать что я умею.'	
)	
@dp.message(Command('help'))	
async def help_cmd(message: types.Message):	
await message.answer(	
'/start — приветствие\n'	
'/help — этот список\n'	
'/time — текущее время\n'	
'Любой текст — получишь обратно'	
)	

@dp.message(Command('time'))	
async def time_cmd(message: types.Message):	
now = datetime.now().strftime('%H:%M:%S %d.%m.%Y')	<i>strftime — форматирование даты/времени</i>
await message.answer(f'Сейчас: {now}')	
@dp.message()	<i>Без фильтра — обрабатывает все остальные</i>
async def echo(message: types.Message):	
await message.answer(f'Ты написал: {message.text}')	<i>message.text — текст входящего сообщения</i>
async def main():	<i>Главная асинхронная функция</i>
await dp.start_polling(bot)	<i>start_polling — запускаем получение обновлений</i>
if __name__ == '__main__':	
asyncio.run(main())	<i>asyncio.run — запускает аsync функцию</i>

⚠ Безопасность токена: Никогда не публикуй TOKEN в открытом коде! Используй: TOKEN = os.getenv('BOT_TOKEN') и файл .env с переменными окружения.

Задание 28.2: Бот-калькулятор с кнопками

★★★

Создай бота-калькулятора:

- при /calc показывает inline-кнопки с цифрами 1-9, 0, операциями и =
 - нажатие кнопки добавляет символ к выражению
 - при нажатии = вычисляет результат через eval()
 - при нажатии C сбрасывает выражение
- Храни состояние каждого пользователя в словаре.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

import asyncio	
from aiogram import Bot, Dispatcher, types, F	<i>F — фильтр для callback_data</i>
from aiogram.filters import Command	
from aiogram.utils.keyboard import InlineKeyboardBuilder	<i>Построитель inline-клавиатур</i>
TOKEN = 'ВАШ_TOKEN'	
bot = Bot(token=TOKEN)	
dp = Dispatcher()	
# Состояние каждого пользователя	

<code>user_expr = {}</code>	<i>dict: user_id -> текущее выражение</i>
<code>def make_keyboard():</code>	<i>Функция создания клавиатуры</i>
<code>builder = InlineKeyboardBuilder()</code>	
<code>buttons = ['7','8','9','+',</code>	
<code>'4','5','6','-'</code>	
<code>'1','2','3','*'</code>	
<code>'C','0','=','/']</code>	
<code>for btn in buttons:</code>	
<code>builder.button(text=btn,</code>	<i>callback_data — данные при нажатии</i>
<code>callback_data=f'calc:{btn}')</code>	
<code>builder.adjust(4)</code>	<i>4 кнопки в ряд</i>
<code>return builder.as_markup()</code>	<i>Возвращаем объект клавиатуры</i>
<code>@dp.message(Command('calc'))</code>	
<code>async def calc_start(message: types.Message):</code>	
<code>uid = message.from_user.id</code>	<i>ID пользователя — уникален</i>
<code>user_expr[uid] = ''</code>	<i>Сбрасываем выражение</i>
<code>await message.answer('Калькулятор:\n0',</code>	<i>Показываем клавиатуру</i>
<code>reply_markup=make_keyboard())</code>	
<code>@dp.callback_query(F.data.startswith('calc:'))</code>	<i>Обработчик нажатий кнопок</i>
<code>async def calc_button(callback:</code>	<i>CallbackQuery — объект нажатия</i>
<code>types.CallbackQuery):</code>	
<code>uid = callback.from_user.id</code>	
<code>btn = callback.data.split(':')[1]</code>	<i>Извлекаем символ после 'calc:'</i>
<code>if btn == 'C':</code>	<i>Очистить</i>
<code>user_expr[uid] = ''</code>	
<code>text = '0'</code>	
<code>elif btn == '=':</code>	<i>Вычислить</i>
<code>try:</code>	
<code>result =</code>	<i>eval() вычисляет строку как</i>
<code>eval(user_expr.get(uid, '0'))</code>	<i>выражение</i>
<code>text = str(result)</code>	
<code>user_expr[uid] = str(result)</code>	<i>Результат становится следующим</i>
	<i>выражением</i>
<code>except:</code>	
<code>text = 'Ошибка'</code>	
<code>user_expr[uid] = ''</code>	
<code>else:</code>	<i>Цифра или операция</i>
<code>user_expr[uid] = user_expr.get(uid, '')</code>	<i>Добавляем к выражению</i>
<code>+ btn</code>	
<code>text = user_expr[uid]</code>	

<code>await callback.message.edit_text(</code>	<i>edit_text — редактируем то же сообщение</i>
<code>f'Калькулятор:\n{text}',</code>	
<code>reply_markup=make_keyboard())</code>	<i>Оставляем клавиатуру</i>
<code>await callback.answer()</code>	<i>Подтверждаем обработку callback</i>
<code>async def main():</code>	
<code>await dp.start_polling(bot)</code>	
<code>asyncio.run(main())</code>	

□ Тест: Тема 28

? Вопрос: Зачем нужен `await` перед `message.answer()`?

А) Это просто синтаксис

Б) Ждём завершения асинхронной операции отправки ← *правильный ответ*

В) Проверяем соединение

Г) Запускаем новый поток

? Вопрос: Что такое `callback_query` в `aiogram`?

А) Текстовое сообщение

Б) Событие нажатия inline-кнопки ← *правильный ответ*

В) Команда `/start`

Г) Медиафайл

Тема 29: GUI-приложения (Tkinter)

Tkinter — встроенная библиотека для создания оконных приложений. Не нужно ничего устанавливать.

Задание 29.1: Конвертер единиц

☆☆☆

Создай GUI-конвертер единиц с помощью Tkinter:

- Combobox с выбором типа конвертации: км↔мили, кг↔фунты, °C↔°F
- поле ввода числа
- кнопка «Перевести»
- метка с результатом

При смене типа конвертации результат сбрасывается.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import tkinter as tk</code>	Основной модуль
<code>from tkinter import ttk</code>	ttk — современные виджеты
<code># Формулы конвертации</code>	
<code>CONVERSIONS = {</code>	Словарь с функциями конвертации
<code> 'км → мили': lambda x: x * 0.621371,</code>	lambda — маленькая анонимная функция
<code> 'мили → км': lambda x: x * 1.60934,</code>	
<code> 'кг → фунты': lambda x: x * 2.20462,</code>	
<code> 'фунты → кг': lambda x: x / 2.20462,</code>	
<code> '°C → °F': lambda x: x * 9/5 + 32,</code>	
<code> '°F → °C': lambda x: (x - 32) * 5/9,</code>	
<code>}</code>	
<code>def convert():</code>	Функция-обработчик кнопки
<code> try:</code>	
<code> value = float(entry.get())</code>	entry.get() — текст из поля ввода
<code> conv_type = combo.get()</code>	combo.get() — выбранный элемент
<code> formula = CONVERSIONS[conv_type]</code>	Берём нужную формулу
<code> result = formula(value)</code>	Применяем формулу
<code> result_label.config(</code>	.config() — меняем свойства виджета
<code> text=f'{value}</code>	split()[0] и [2] — первое и третье слово
<code>{conv_type.split()[0]} = {result:.4f}</code>	
<code>{conv_type.split()[2]}',</code>	
<code> fg='green')</code>	fg — цвет текста
<code> except ValueError:</code>	Если ввели не число
<code> result_label.config(text='Ошибка:</code>	
<code>введи число!', fg='red')</code>	
<code>def on_combo_change(event):</code>	При смене типа конвертации
<code> result_label.config(text='')</code>	Сбрасываем результат

<code>entry.delete(0, tk.END)</code>	<i>Очищаем поле ввода (0 — начало, END — конец)</i>
<code># Создаём главное окно</code>	
<code>root = tk.Tk()</code>	<i>Главное окно</i>
<code>root.title('Конвертер единиц')</code>	
<code>root.geometry('420x200')</code>	<i>Ширина x Высота в пикселях</i>
<code>root.resizable(False, False)</code>	<i>Запрещаем изменение размера</i>
<code>root.configure(bg='#f0f0f0')</code>	<i>Цвет фона</i>
<code># Виджеты</code>	
<code>ttk.Label(root, text='Тип конвертации:').pack(pady=(20,5))</code>	<i>pack() — размещает виджет</i>
<code>combo = ttk.Combobox(root, values=list(CONVERSIONS.keys()), width=20, state='readonly')</code>	<i>state='readonly' — только выбор из списка</i>
<code>combo.current(0)</code>	<i>Выбираем первый элемент по умолчанию</i>
<code>combo.pack()</code>	
<code>combo.bind('<<ComboboxSelected>>', on_combo_change)</code>	<i>bind — привязываем событие к функции</i>
<code>entry = ttk.Entry(root, width=25, font=('Arial', 13))</code>	<i>Поле ввода</i>
<code>entry.pack(pady=10)</code>	<i>pady — отступ по вертикали</i>
<code>ttk.Button(root, text='Перевести', command=convert).pack()</code>	<i>command=convert — функция при нажатии</i>
<code>result_label = tk.Label(root, text='', font=('Arial', 13), bg='#f0f0f0')</code>	<i>Метка для результата</i>
<code>result_label.pack(pady=10)</code>	
<code>root.mainloop()</code>	<i>Запускаем цикл обработки событий</i>

□ **Результат:** Открывается окно приложения с выпадающим списком, полем ввода и кнопкой. После ввода числа и нажатия кнопки — результат появляется зелёным.

⊖ Частая ошибка:

✗ `label = tk.Label(root, text=result)` # не обновится!

✓ `label.config(text=result)` # обновляет существующий виджет

Создание нового Label не обновляет старый — появится второй. Для обновления текста существующего виджета используйте `.config(text=новый_текст)`.

□ **Тест: Тема 29**

? Вопрос: Что делает `root.mainloop()`?

А) Создаёт главное окно

Б) Запускает бесконечный цикл обработки событий GUI ← *правильный ответ*

В) Закрывает приложение

Г) Рисует виджеты

? Вопрос: Как обновить текст существующего Label?

А) Создать новый Label

Б) `label.text = 'новый'`

В) `label.config(text='новый')` ← *правильный ответ*

Г) `label.update('новый')`

☐ Итоговая таблица прогресса

Тема	Заданий	Тест	Отметка
Тема 23: Регулярные выражения	3	3 вопр.	_____
Тема 24: Веб-скрапинг	2	2 вопр.	_____
Тема 25: Flask	2	2 вопр.	_____
Тема 26: Pandas	2	2 вопр.	_____
Тема 27: Matplotlib	1	2 вопр.	_____
Тема 28: Telegram-боты	2	2 вопр.	_____
Тема 29: Tkinter	1	2 вопр.	_____

☐ Сборник 1

☐ Сборник 2

☐ Сборник 3

☒ Сборник 4 ✓

☐ **Весь курс пройден!** Поздравляем! Ты прошёл все 4 методички и 4 сборника заданий. Итого: 87 заданий и 69 вопросов теста. Теперь бери реальный проект и вперёд — на GitHub!